FULL-DAY TRAINING · DECK 1 OF 2

SDLC, Methodologies & Why Testing Matters

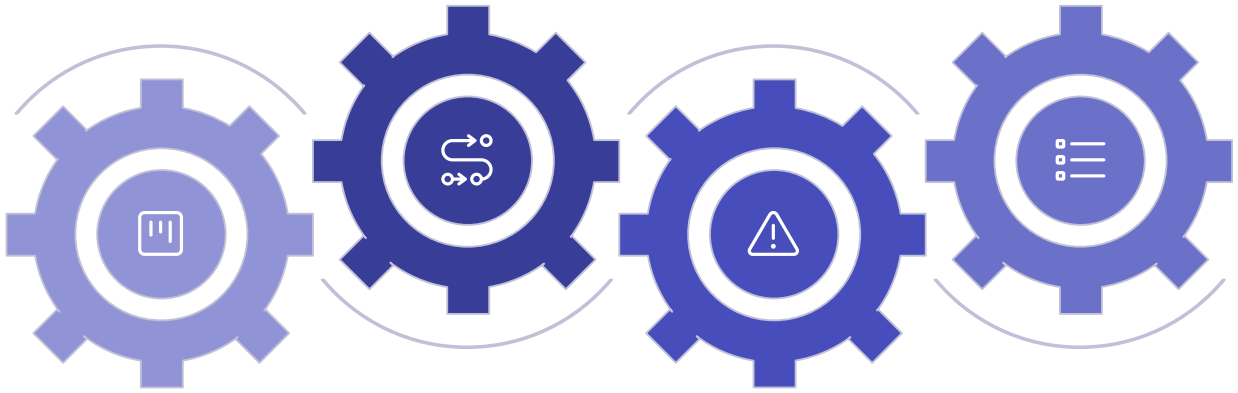
AM Session · 08:15–12:30 · Sustain Engineering Foundations

Methodologies

Waterfall, Agile, DevOps approaches

AM Recap

Key takeaways and action items



SDLC Stages

Plan, design, build, test, deploy

Risk of Inadequate Testing

Defects, delays, increased support costs

OPENING QUESTION

We know how to write code.

But how does a business idea actually become production software?

Today's session answers that question — step by step.

MONEY TRANSFER SYSTEM · LIGHT HOOK

Business says:

> "Customers should be able to transfer money."

Is that enough to start coding?

What we have

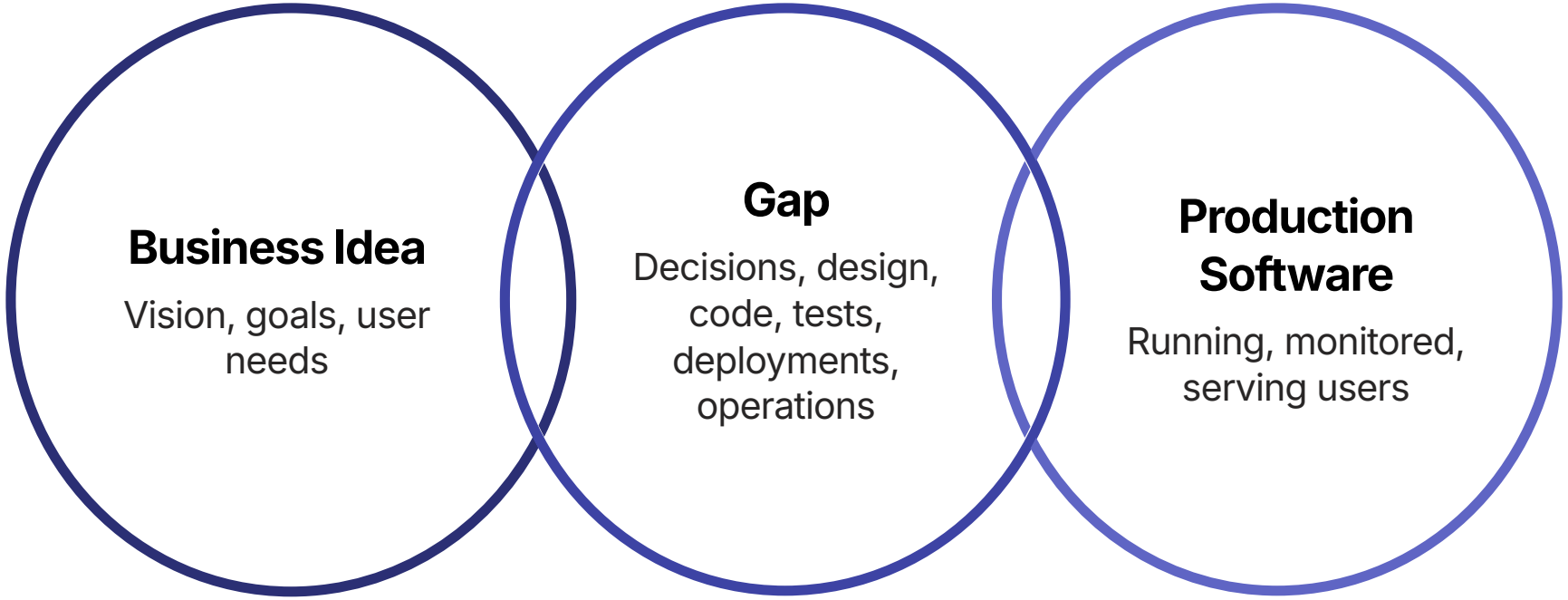
One sentence. A business desire.

What we need

Rules, constraints, design, security, testing, operations...

THE GAP

From Idea to Production — There's a Lot in Between



Every step in that gap needs structure. Without it — teams guess, quality suffers, production breaks.

INTRODUCING SDLC

Software Development Life Cycle

A structured journey — from a business need to a system in operation.

Structured

Every stage has a purpose and outcome

Repeatable

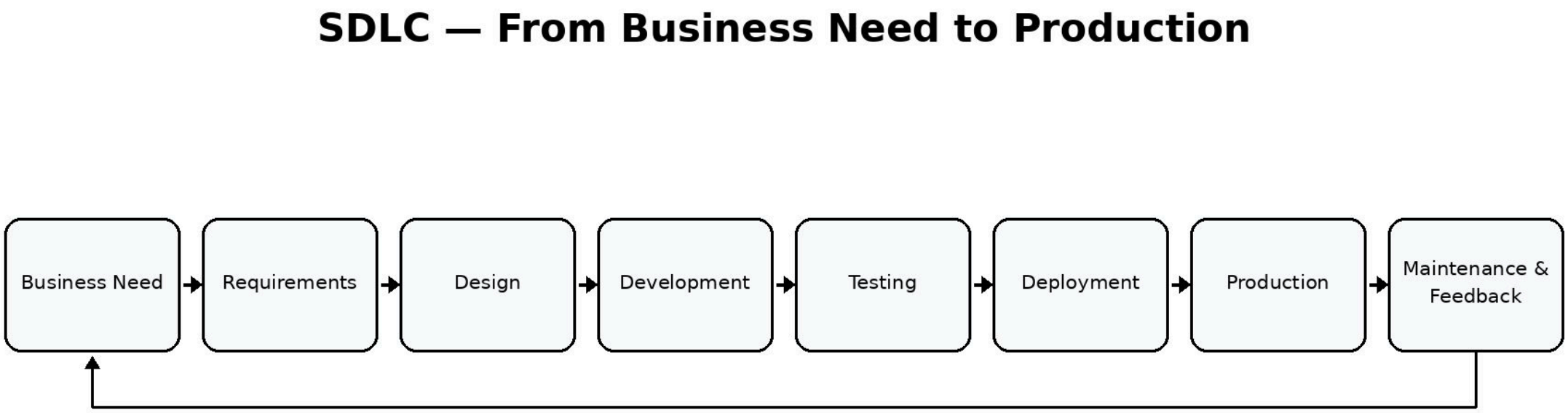
Teams can follow it project after project

Adaptable

Different methodologies execute it differently

CANONICAL DIAGRAM A

The Full SDLC Lifecycle — At a Glance



The loop closes: feedback and new needs restart the cycle.

THINK ABOUT IT

Why can't teams just… code it?

What goes wrong when there's no structured lifecycle?

Three Reasons Organizations Need a Lifecycle

Alignment

Business, developers, testers and operations all work toward the same goal

Predictability

Stages create checkpoints — progress and risk are visible

Quality & Risk Control

Problems are caught earlier, not after users encounter them in production

STAGE 1

Business Need / Planning

What problem are we solving?

Define the problem

Understand why the software is needed before deciding what to build

Scope the effort

What is in? What is out? Who are the stakeholders?

Assess feasibility

Is this achievable within business constraints?

Key output

Project charter, scope statement, or a clear problem definition

🏛 MTS EXAMPLE · STAGE 1

Business Outcome, Not Implementation

> "Enable customers to safely transfer money between accounts."

✅ Business outcome

Safe money transfer — a capability that delivers value

❌ Not yet

No database chosen. No API defined. No UI wireframe.

That comes later.

PLANNING · FEASIBILITY

Four Feasibility Questions

1

Business

Does solving this problem create enough value to justify the investment?

2

Technical

Do we have (or can we get) the technology and skills needed?

3

Operational

Can the organization actually run and support this system?

4

Schedule / Cost

Is the timeline and budget realistic?

STAGE 2

Requirements

Define *WHAT* the system must do — before deciding *HOW*.

Requirements answer

What should the system do? For whom? Under what conditions?

Why this stage exists

Vague requirements produce wrong designs — no amount of clever code fixes a wrong spec

Deep dive on requirement documents (SRS, BRD, FRD) comes in Deck 2 — PM session.

REQUIREMENTS · PREVIEW

Two Types of Requirements

Functional

What the system does

Features, behaviors, business rules

Example: The system shall allow a user to initiate a transfer.

Non-Functional

How well the system does it

Performance, security, availability, scalability

Example: Transfer response time shall be under 3 seconds.

Both types matter. Both need to be tested. We'll go deeper in Deck 2.

REQUIREMENTS · PROBLEM

Spot the Problem

> "The transfer should be fast."

Is this requirement testable — or ambiguous?

Fast for whom?

100ms? 3 seconds? 30 seconds? Nobody agrees.

Test on what?

No measurable target → no pass/fail criterion → no meaningful test

Result in production

Developer builds "fast." Tester tests "fast." User expects "fast." Three different things.

REQUIREMENTS · BETTER

Make It Measurable

❌ Ambiguous

"The transfer should be fast."

✅ Testable

"A domestic transfer shall complete within 3 seconds under normal load (up to 1,000 concurrent users)."

Why it matters

- A tester can design an exact test
- A developer knows the acceptance bar
- A business stakeholder can verify the value was delivered

Clarity in requirements is the first line of quality defense.

STAGE 3

Design

Translate requirements into a solution structure.

Input

Approved, clear requirements

Output

Architecture, component diagrams, data models, interface specifications

Key rule

Design decisions made here determine how easy or hard testing and maintenance will be

High-Level vs. Detailed Design

High-Level Design

Architecture & components
What are the major pieces? How do they connect?

→ System topology, major services, database strategy, external integrations

Detailed Design

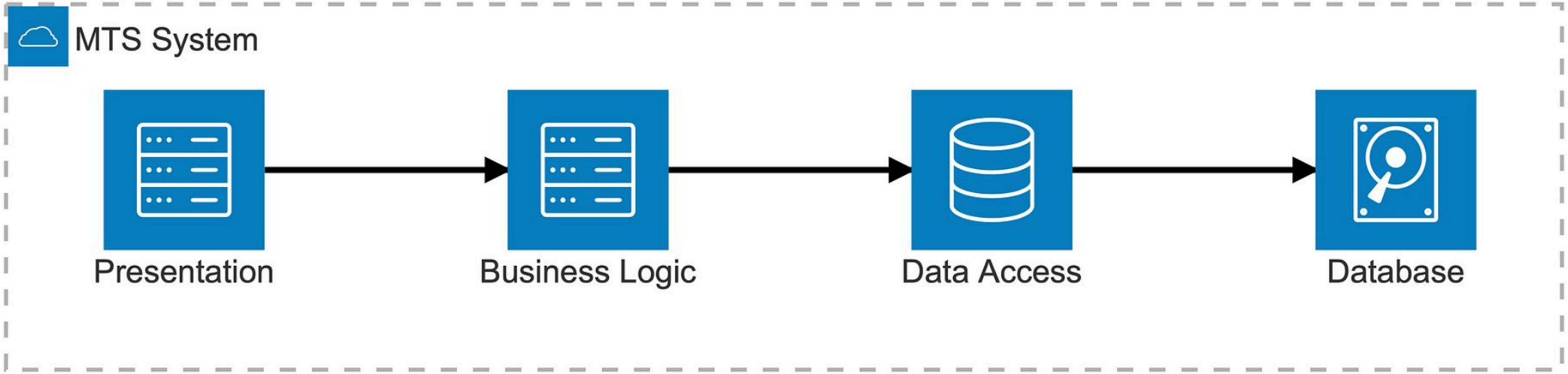
Implementation blueprint
How is each component built internally?

→ Class diagrams, API contracts, database schemas, algorithm logic

Testers use design documents to plan test strategies — not just to understand the code after it's written.

MTS EXAMPLE · DESIGN

Simple System Structure — MTS



Four components. Clear responsibility boundaries. This structure guides both developers and testers — what to build and where to verify.

STAGE 4

Development

Design becomes working code.

Build to spec

Developers implement features against agreed requirements and design

Integrate continuously

Code is merged frequently — integration problems surface early, not at the end

Collaborate actively

Questions about requirements or design get answered now — not after testing fails

Developer Responsibilities Beyond Writing Code

Unit Checks

Test individual components in isolation before handing off

Code Review

Peers review logic, security, and maintainability — another quality gate

Build Discipline

Broken builds block the whole team; keep the build green

Documentation

Inline comments, API docs — future testers (and yourself) will thank you

THINK ABOUT IT

Code compiles. Build passes.

Is the feature ready?

Compilation confirms syntax. It says nothing about behaviour, correctness, or edge cases.

STAGE 5

Testing

Verify expected behaviour. Uncover risk before users do.

Not just "find bugs"

Testing provides **risk information** to the team — what is safe to release and what is not

Every stage, not just at the end

Modern teams test throughout the lifecycle — not only after development completes

Testers need the requirements

You can only verify behaviour if you know what the correct behaviour should be

Verification vs. Validation

Verification

"Are we building it right?"

Does the product match its specification and design?
→ Code reviews, unit tests, design walkthroughs

Validation

"Are we building the right thing?"

Does the product deliver what the business actually needs?
→ Acceptance testing, user demos, exploratory testing

Different organizations use these terms slightly differently — what matters is the intent behind both questions.

Testing Is More Than Bug-Finding

Confidence

Stakeholders need evidence — not just the developer's word — that the system works

Risk Information

Testing reveals what is unknown and unsafe to release — informing a release decision

Requirement Validation

Tests confirm that what was specified was actually implemented — closing the requirements loop

STAGE 6

Deployment

Move a tested version toward users — in a controlled way.

1

Build Artifact

Tested, approved version packaged for release

2

Staging / Pre-Prod

Final smoke tests in a production-like environment

3

Production

Software runs where real users access it

Deployment ≠ Release

Deployment

Placing the software into an environment

→ The code is **there** — on a server, in a container

Release

Making a capability **available to users**

→ A feature flag turns it on; a rollout makes it visible

In production sustaining, you'll often deploy a fix and release it to a subset of users first — to control blast radius.

STAGE 7

Production / Operations

Real users. Real data. Real dependencies. Real failures.

Real Users

Behavior in production rarely matches what was tested — diversity of devices, data, usage patterns

Real Data

Production data volumes and edge cases exceed what any test environment replicates

Real Failures

Network timeouts, third-party outages, resource exhaustion — things tests rarely simulate fully

STAGE 8

Maintenance

Corrective

Bug fixes, incident resolution, patches for discovered defects

Adaptive

Changes driven by business, regulatory, or platform changes

Perfective

Performance improvements, security hardening, scalability upgrades

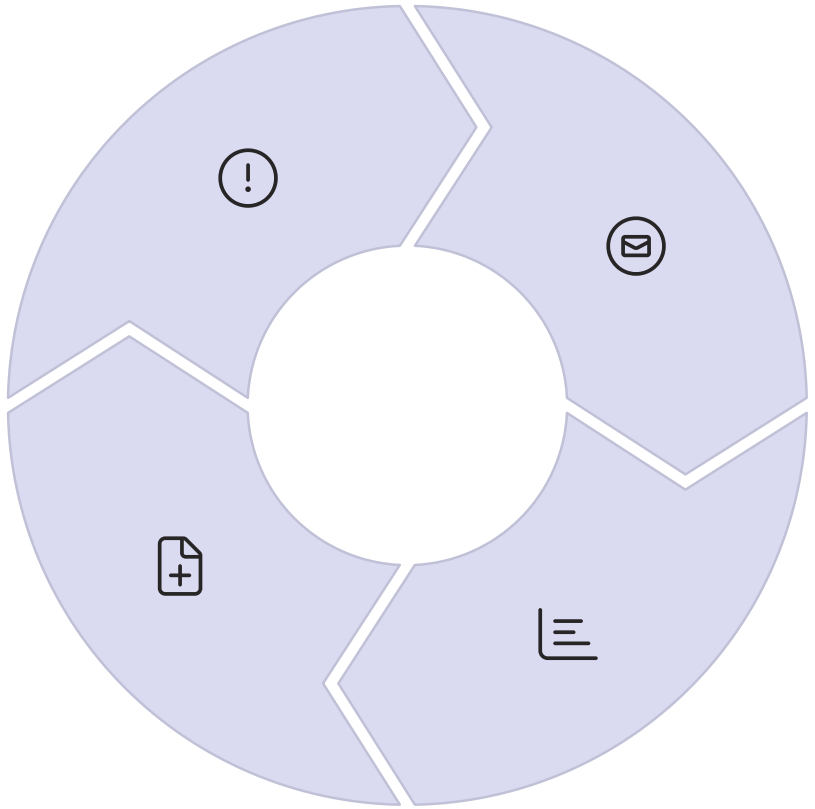
As sustain engineers, **most of your daily work lives here** — fixing and improving software already in production.

THE LOOP CLOSES

Feedback Is a New Beginning

Incidents

Production failures expose gaps in requirements, design, or testing



User Feedback

Users reveal unmet needs and unexpected usage patterns

New Requirements

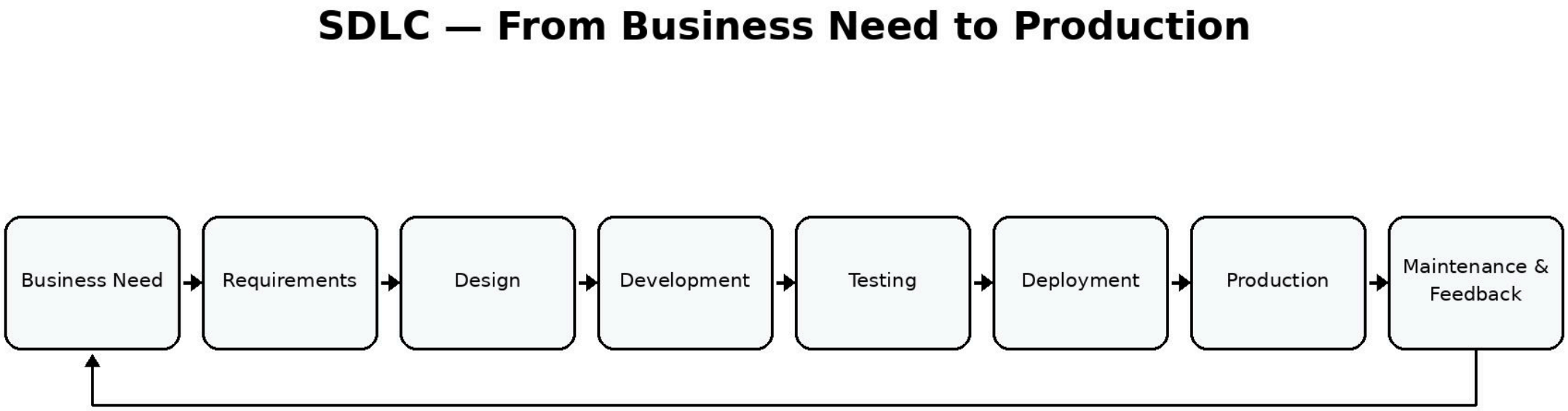
Insights re-enter the SDLC — the cycle continues

Analytics

Operational data surfaces performance, error rates, adoption signals

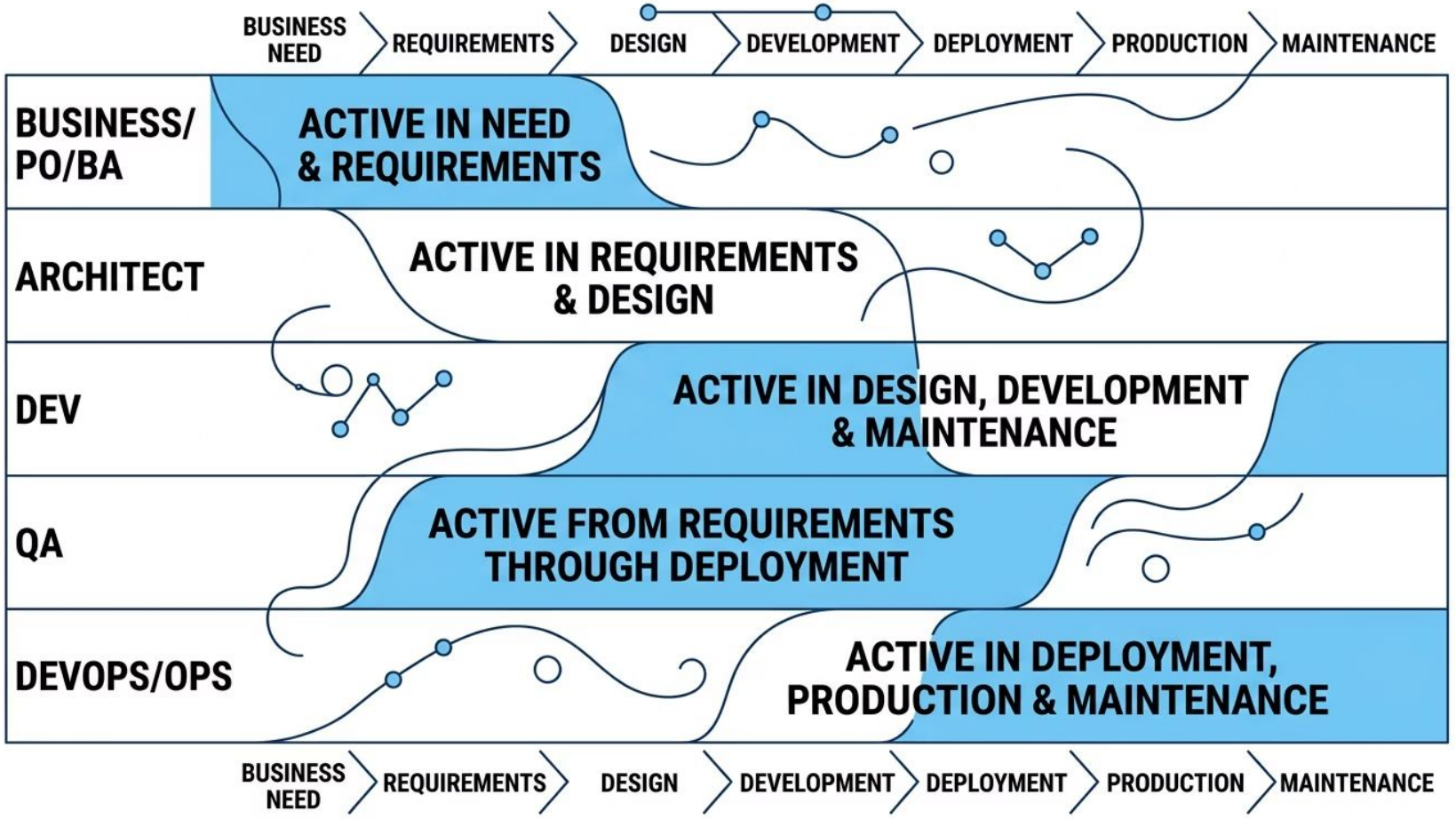
CHECKPOINT

The Full SDLC — Now You Explain It



Turn to a partner. Walk through each stage. State the stage name, its purpose, and its key output.

Who Participates — and When?



No rigid handoffs. Modern teams collaborate across stages — especially QA, who engages from requirements, not just after coding.

THINK ABOUT IT

Same lifecycle stages.

Do all organizations execute them the same way?

SDLC METHODOLOGY

Methodology = How Teams Organize and Execute the Lifecycle

Same stages. Different rhythms.

All methodologies move from need → design → build → test → deliver. The **sequence, speed, and feedback structure** differ.

- When do teams get feedback?
- How much does change cost?
- How often does working software ship?

Three models we'll cover

Waterfall → V-Model → Agile

Each emerged to solve a different problem in software delivery.

METHODOLOGY 1 · WATERFALL

Waterfall — The Problem It Solves

When might a sequential, phase-by-phase approach make sense?

Requirements are stable

What to build is fully understood before work begins

Sequential handoffs are acceptable

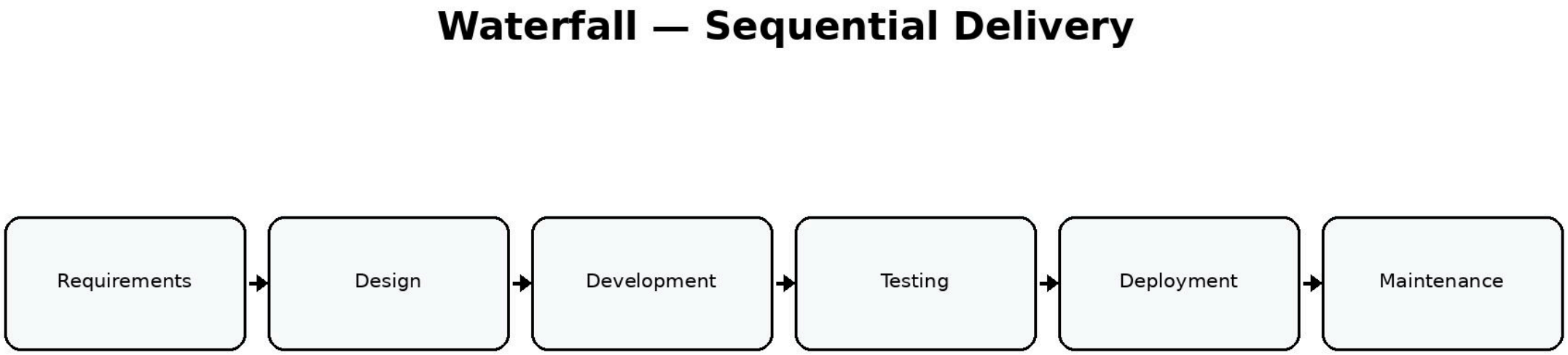
One phase completes and signs off before the next starts

Controlled, auditable process required

Regulated industries (finance, aerospace, medical) often need documented phase gates

CANONICAL DIAGRAM B

Waterfall — Sequential Phases



Each phase produces a deliverable. The next phase does not begin until the previous one is formally complete.

Waterfall Strengths

Clear Documentation

Each phase produces formal artifacts — easier to audit, transfer, or hand over

Defined Phase Gates

Entry/exit criteria make progress visible and decisions explicit

Predictable Planning

When scope and requirements are stable, timelines and budgets can be estimated with confidence

Waterfall is not inherently bad — it fits contexts where stability and auditability matter more than speed of change.

Waterfall Challenges

Late feedback loop

Testing happens after development is complete — problems surface late in the project lifecycle

Change is expensive

A requirement change after design is signed off means rework across multiple completed phases

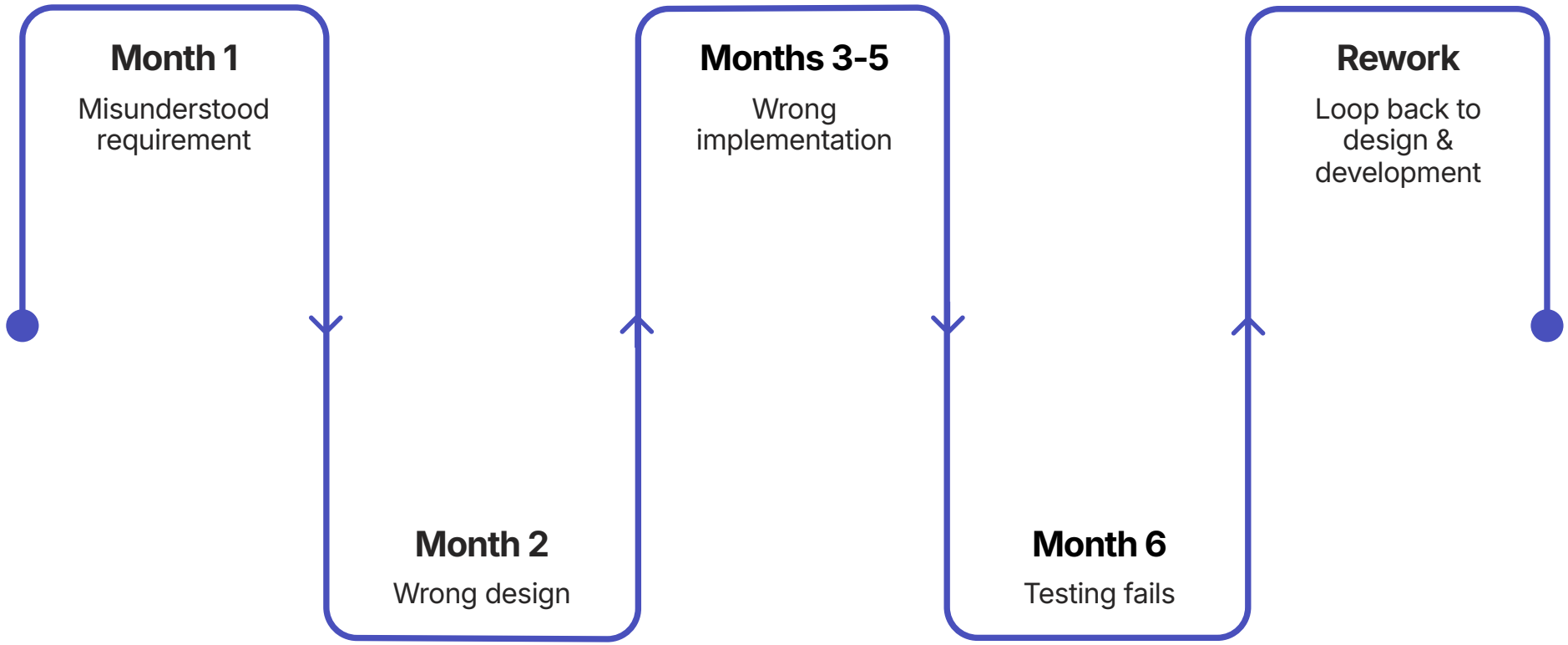
Users see software late

No working increment until near project end — misalignment with user expectations discovered very late

SCENARIO

The Month-1 Mistake

A requirement is misunderstood in Month 1. Nobody notices until testing in Month 6.



The longer a defect sits undetected, the more decisions — and code — are built on top of it. Rework grows.

METHODOLOGY 2 · V-MODEL

Introducing the V-Model

What if testing activities were planned *alongside* each development stage — not after?

The core idea

For every development stage going down the left side of the V, there is a corresponding test stage on the right side

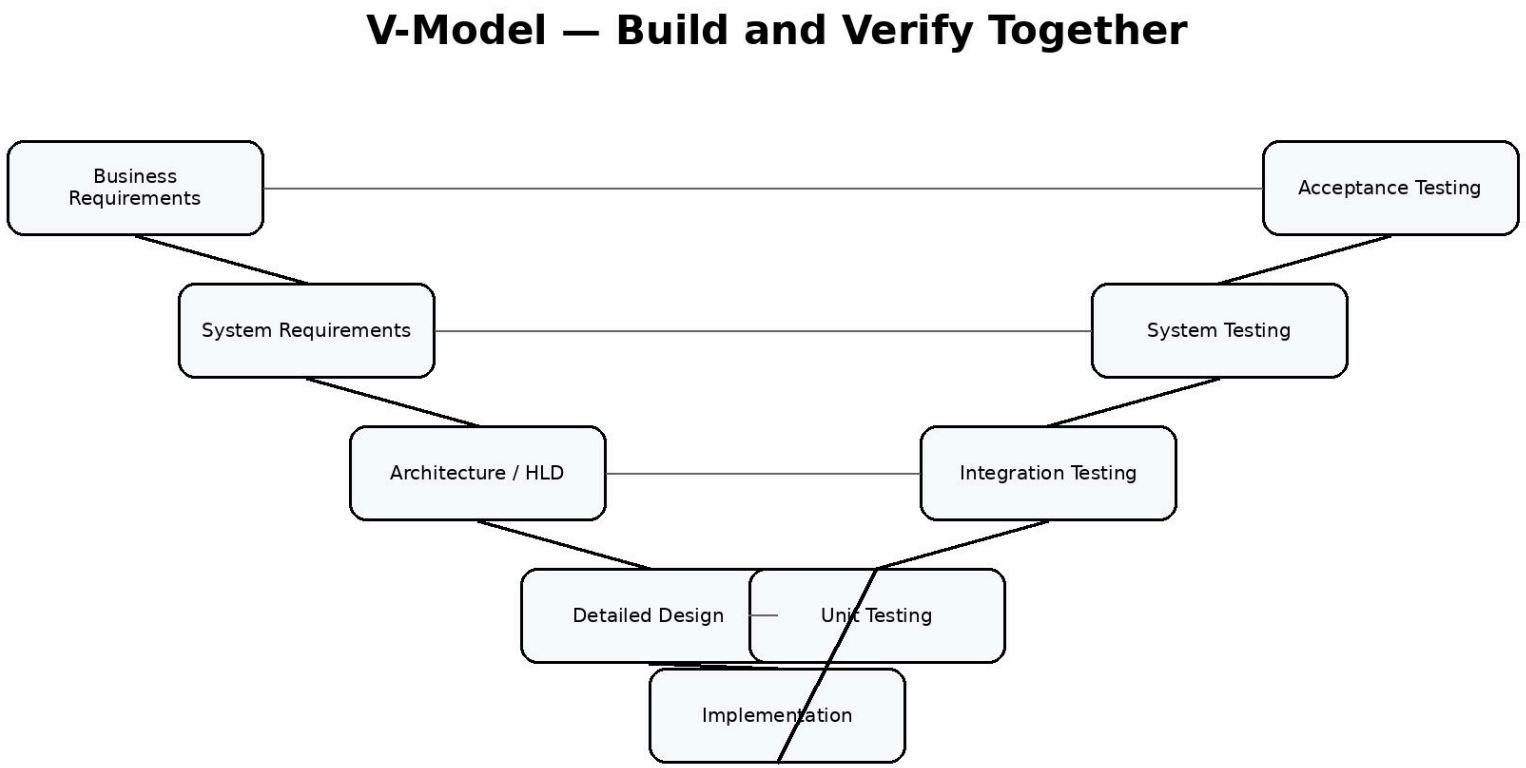
Why it matters

Test planning begins during requirements — not after the code is written.

Problems in specs are caught before implementation

CANONICAL DIAGRAM C

V-Model — Development Meets Testing



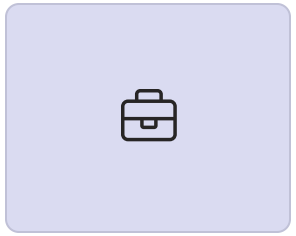
Left side — Decomposition

Requirements broken down into increasingly specific design decisions

Right side — Verification

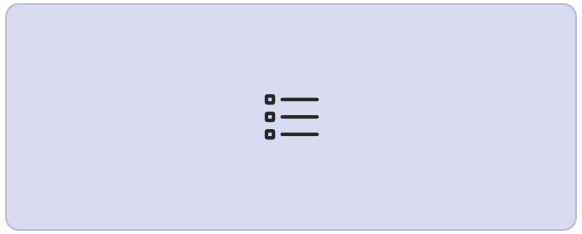
Each test level validates the corresponding design decision

Walking the Left Side



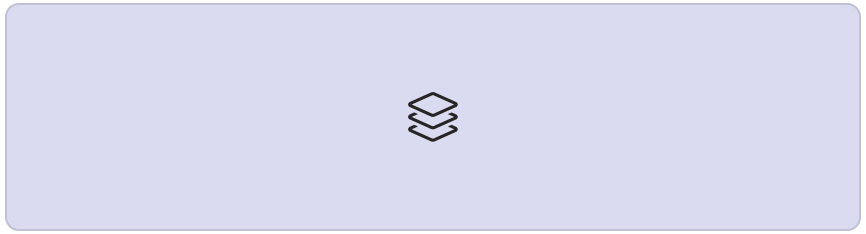
Business Requirements

What the business needs — in business language



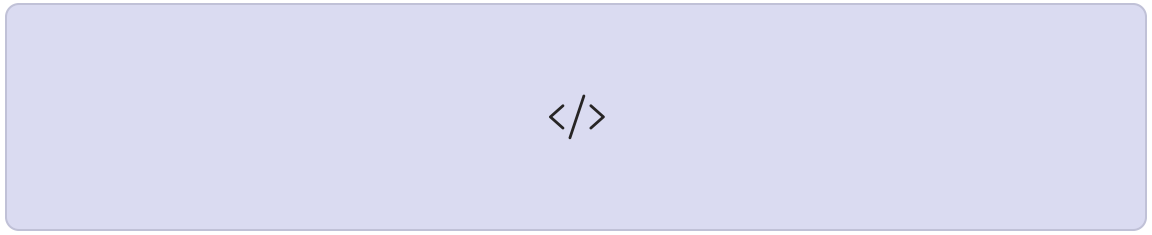
System Requirements

Functional + non-functional specification of the system



Architecture / High-Level Design

Major components, interfaces, and system structure



Detailed Design → Implementation

Module-level design and working code

Walking the Right Side

Unit Testing

Verifies individual modules against their detailed design

Integration Testing

Verifies components interact correctly — matches architecture design

System Testing

Verifies the end-to-end system against system requirements

Acceptance Testing

Validates the system fulfills business requirements — user-facing sign-off

The Mapping Insight

Development Stage	Test Stage	Key Question
Business Requirements	Acceptance Testing	Does it meet business needs?
System Requirements	System Testing	Does it meet the spec?
Architecture / HLD	Integration Testing	Do components work together?
Detailed Design	Unit Testing	Does the module work correctly?

Test plans at each level are written when the corresponding design is written — not after coding finishes.

V-Model — Benefit and Limitation

✅ Benefit

Strong test alignment: every requirement has a planned test.

Early test planning catches ambiguous or incomplete specs before development.

⚠️ Limitation

Still relatively sequential: like Waterfall, it assumes requirements are stable. Late-stage changes are still costly to absorb.

The V-Model is a step forward in test thinking — but it still struggles when requirements evolve frequently.

THINK ABOUT IT

What if requirements change every few weeks?

Does a sequential, phase-gated model still work?

METHODOLOGY 3 · ITERATIVE & INCREMENTAL

Build in Smaller Pieces. Learn. Improve.

Iterative thinking

Build something. Get feedback. Refine the same capability in the next cycle.

Incremental thinking

Add usable slices of functionality. Each increment delivers real value to users.

The key shift

Feedback loops shorten dramatically — mistakes are discovered in weeks, not months.

Iterative vs. Incremental — What's the Difference?

Iterative

Build a rough version → get feedback → improve the same feature
→ Each cycle **refines** what already exists

Incremental

Build feature A → ship it → build feature B → ship it
→ Each increment **adds** new usable capability

Modern teams combine both — they add new features *and* improve existing ones within the same short cycles.

AGILE

Agile: Short Feedback Cycles + Frequent Working Increments

Agile is values + principles

The Agile Manifesto (2001) defines priorities — individuals, working software, collaboration, responding to change

Not one method

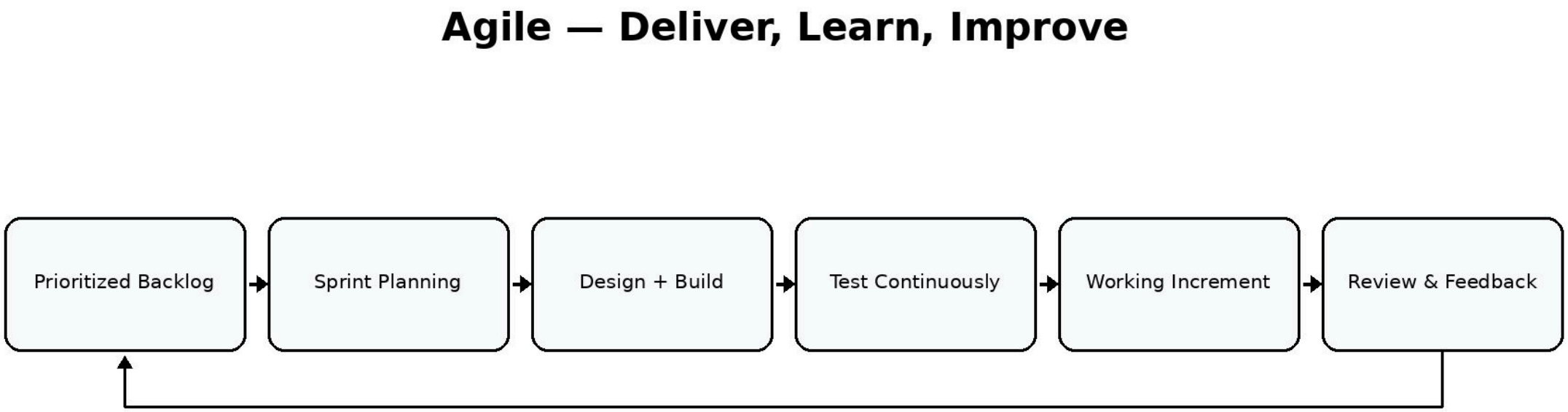
Agile describes **how to think** — Scrum, Kanban, SAFe are frameworks that implement Agile values. Agile ≠ Scrum.

What changes

Requirements evolve through collaboration. Working software delivered in short cycles. Feedback drives what comes next.

CANONICAL DIAGRAM D

The Agile Loop — Continuous Delivery Cycle



Where Is Testing in Agile?

Everywhere.

NOT a separate final phase

- Testing happens inside every sprint
- Testers collaborate with developers from day one of each increment
- Automated tests run on every code commit

The goal

Every sprint produces a **tested, potentially shippable** increment — not "code done, testing next sprint"

AGILE IN PRACTICE · SCRUM CONTEXT

Scrum — One Agile Framework

Product Backlog

Ordered list of everything the product needs — owned by the Product Owner

Sprint

Time-boxed iteration (typically 1–4 weeks) — a fixed period to deliver a working increment

Increment

Working, tested software delivered at the end of each sprint — real progress, not just reports

Scrum is context only. This is not a Scrum certification. The point: Agile frameworks give teams a structured way to apply iterative + incremental thinking.

Waterfall vs. V-Model vs. Agile

Dimension	Waterfall	V-Model	Agile
Change tolerance	Low	Low–Medium	High
Feedback timing	Late (post-build)	Planned early	Continuous
Testing involvement	End phase	Planned throughout	Every sprint
Delivery cadence	End of project	End of project	Each sprint
Best fit	Stable, regulated	Safety-critical systems	Evolving requirements

MYTH BUSTING · AGILE

Agile Does Not Mean...

❌ No documentation

Agile values *working software over comprehensive documentation* — not no documentation

❌ No planning

Agile teams plan every sprint. Backlogs, priorities, and acceptance criteria are all planning artifacts

❌ No testing

Testing is more intensive in Agile — it happens inside every sprint, not at the end of a project

Agile done poorly looks like chaos. Agile done well is highly disciplined — just in shorter cycles.

DEVOPS BRIDGE

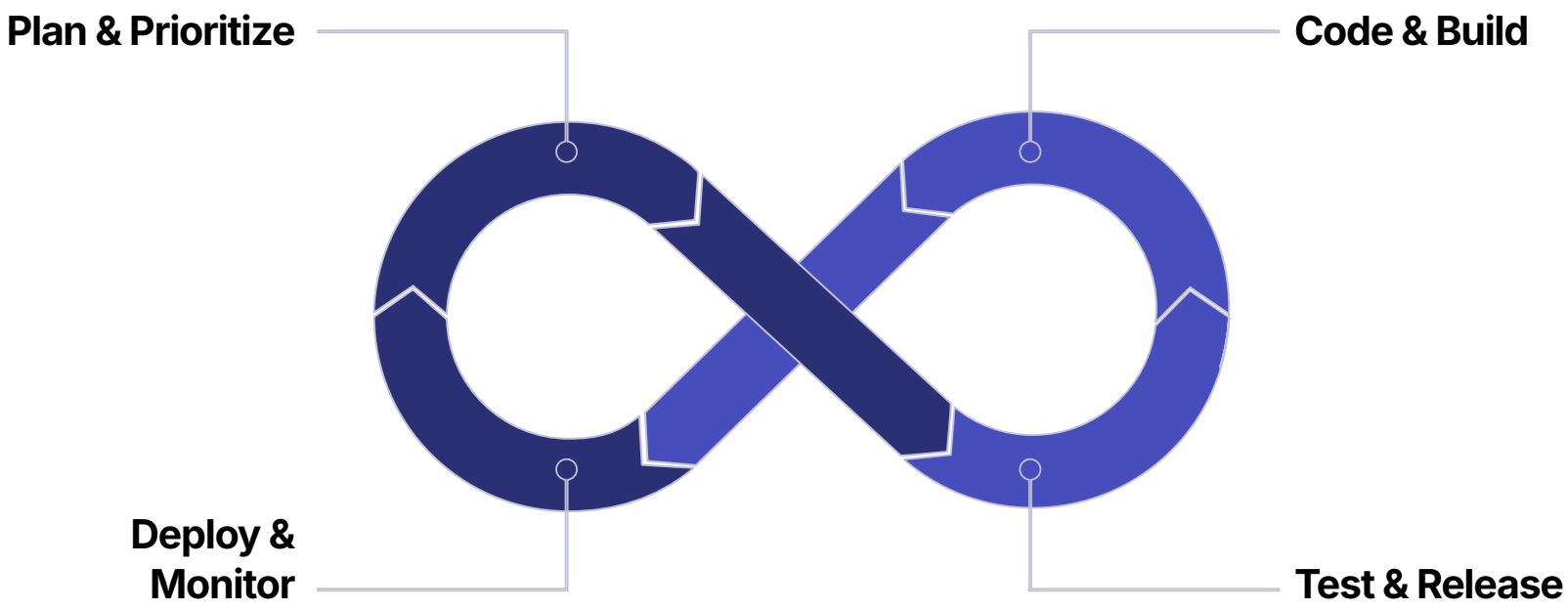
DevOps — Development and Operations, Together

What DevOps changes

- Dev and Ops collaborate throughout — not handoff at deployment
- Automated pipelines: build → test → deploy → monitor
- Faster, safer delivery of tested software to production

Important clarity

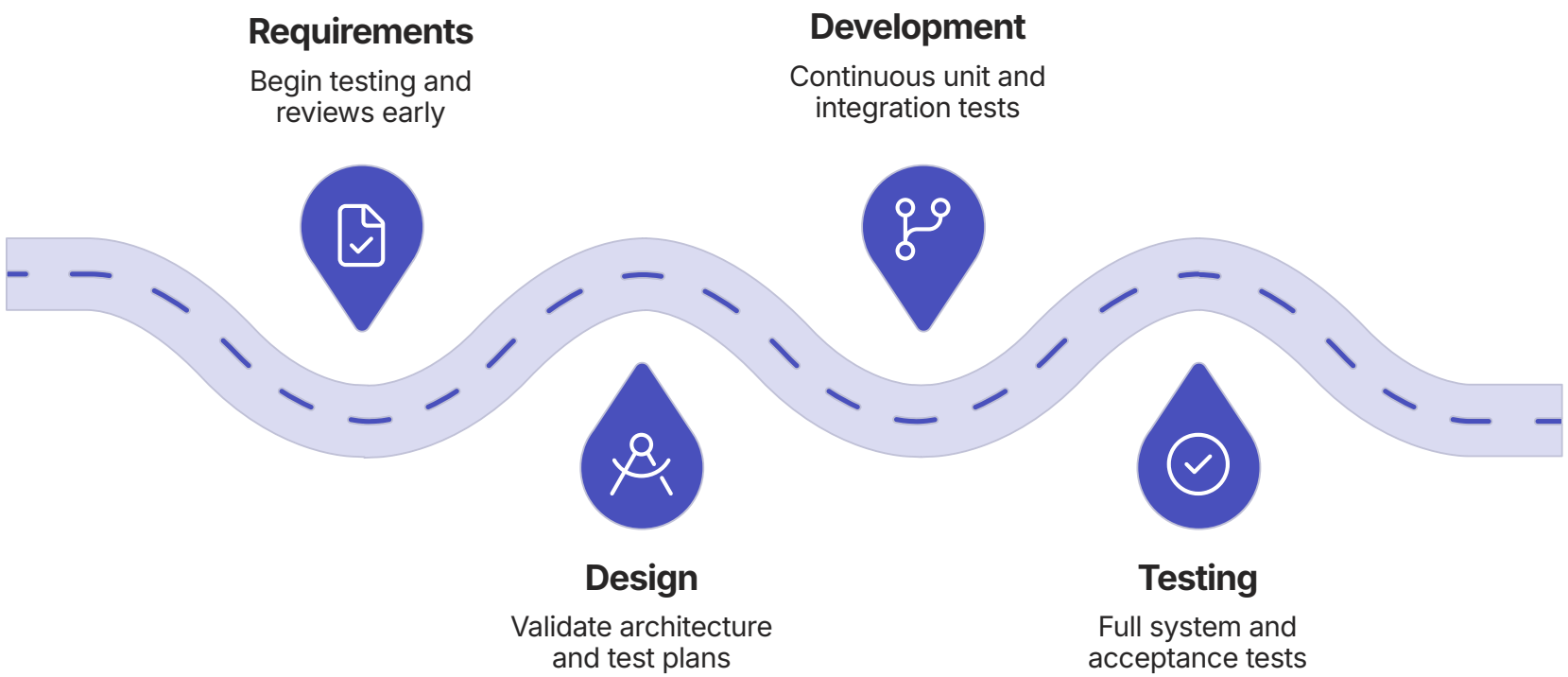
DevOps does **not replace** SDLC or Agile — it bridges development practice with operational delivery and feedback



SHIFT-LEFT

Shift-Left — Find Problems Sooner

Move reviews, testing, and security checks earlier in the lifecycle.



Why earlier is better

A defect found during requirements review costs a conversation. The same defect found in production costs an incident.

Practical examples

Review requirements before design begins. Write test cases when requirements are written. Static code analysis during development.

SHIFT-RIGHT

Shift-Right — Learn From Production

Pre-production testing cannot replicate everything. Production data teaches you things labs cannot.

Monitoring

Observe real user behavior, error rates, and performance in production continuously

Canary / Feature Flags

Release to a small user group first — validate in production at controlled scale

Feedback Loop

Production observations become requirements — shift-right and shift-left work together

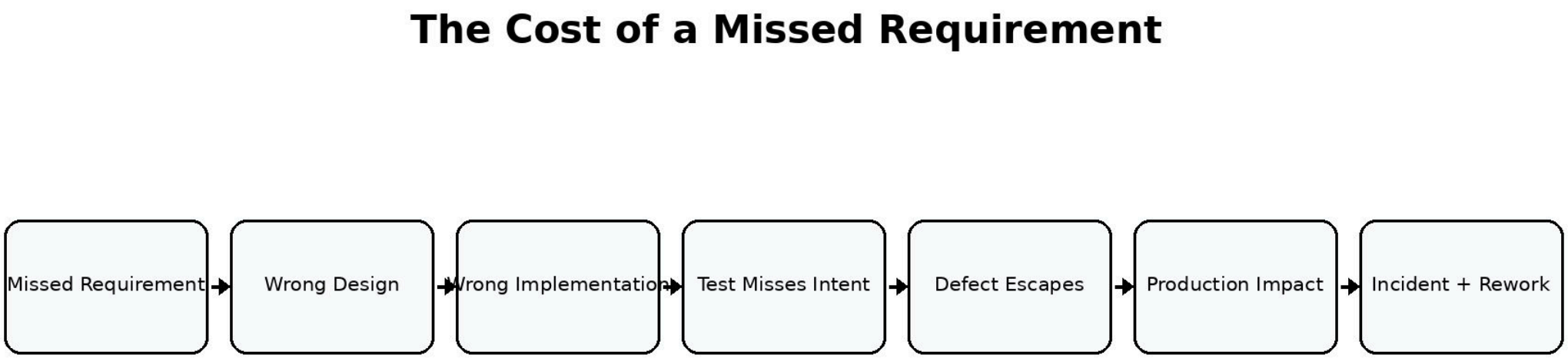
CRITICAL QUESTION

What happens when testing is inadequate?

Let's trace a single missed requirement all the way to production.

CANONICAL DIAGRAM E

The Risk Chain — One Missed Requirement



Each stage amplifies the mistake. What could have been a 5-minute conversation in requirements becomes a production incident and business cost.

Six Impacts of Inadequate Testing

Customer Trust

Defects in production damage user confidence — often permanently

Business Impact

Revenue loss, SLA breaches, regulatory exposure

Data Integrity

Corrupt transactions or missing records can be very hard to reverse

Security

Unverified code paths create exploitable vulnerabilities

Availability

Performance failures under real load cause outages impacting all users

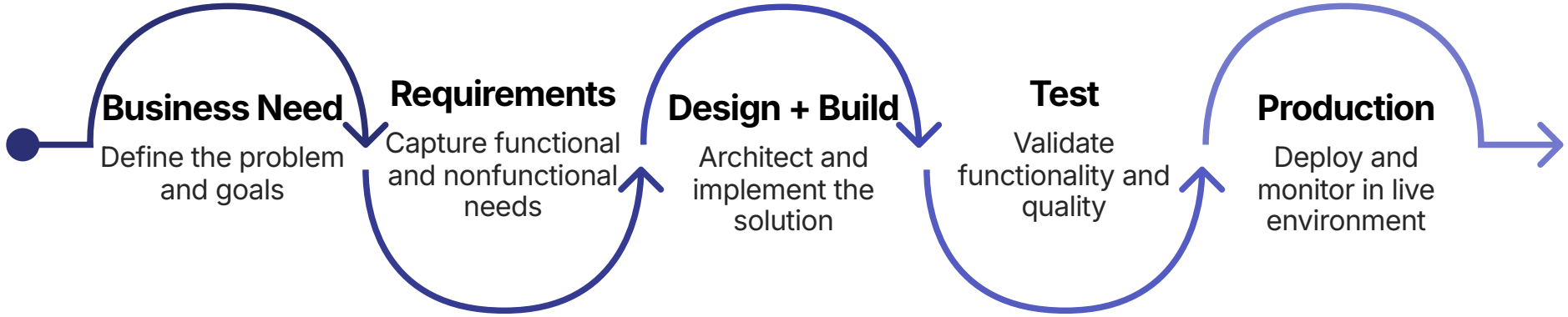
Cost & Reputation

Escaped defects cost more to fix — and reputation damage outlasts the incident

Key insight: the cost and impact of a defect generally grows the later it escapes. Finding problems early — through testing — is not overhead. It is risk management.

AM RECAP · BRIDGE TO DECK 2

What We Covered This Morning



You can now explain:

- Every stage of a common SDLC model
- Waterfall, V-Model, and Agile — tradeoffs and fit
- Where testing fits in each methodology
- Why inadequate testing becomes production and business risk

Final question before lunch

"Testing starts with understanding *what* should be built."

So where do testers get that truth?

→ Requirements · BRD · FRD · SRS · STLC · Environments

Deck 2 · PM Session · 13:30